Reinforcement Learning

Foundations of Practice-oriented AI

Mengyan Zhang

School of Computer Science
University of Bristol

About me



Research: sequential decision-making under uncertainty – reinforcement learning · bandits · Bayesian optimisation · active learning · experimental design · AI for public health

Roadmap

1. The RL problem: MDPs, values, Bellman
2. Three families of algorithms: value, policy, model
3. RL meets large language models
4. RL in practice and open questions

Aim: the **core ideas** you need to read RL papers, and a map of where the field is going – not a deep dive into one method.

Why care about RL? A decade of milestones

DQN
Atari from pixels
2015

2016
AlphaGo
beats Lee Sedol

AlphaZero
chess, shogi, Go by self-play
2018

2020
MuZero
learns its own model

Tokamak plasma
control
2022

2022
RLHF:
InstructGPT/ChatGPT

RL-trained
reasoning LLMs
2024–25


DQN


AlphaGo


AlphaZero


TCV tokamak

Same recipe throughout:
act, *observe consequences*, *improve*.

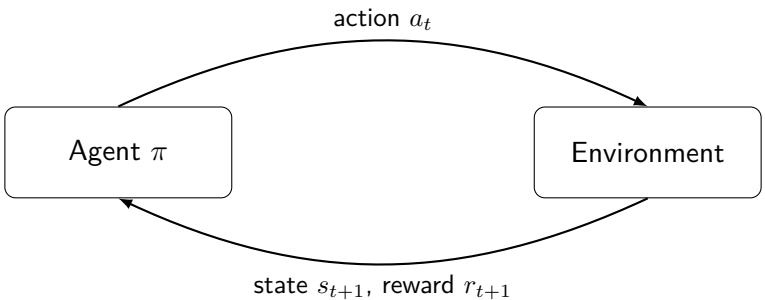
Mnih et al. 2015; Silver et al. 2016, 2018; Schrittwieser et al. 2020; Degrave et al. 2022; Ouyang et al. 2022; DeepSeek-AI 2025. Photos (Wikimedia Commons): Evan-Amos (PD); Donarreiskoffer (CC BY-SA 3.0); Wilfredor (CC0); CRPP-EPFL (CC BY-SA 2.5).

Roadmap

1. **The RL problem: MDPs, values, Bellman**
2. Three families of algorithms: value, policy, model
3. RL meets large language models
4. RL in practice and open questions

The RL loop

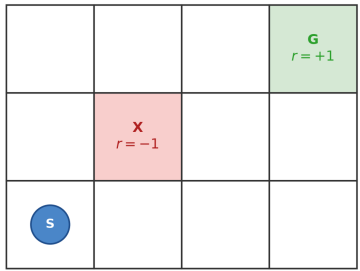
An **agent** takes **actions** in an **environment**, observes a new **state** and a **reward**, and repeats. The goal is to maximise total reward received – by *trying* actions, not by being told which is best.



The formal model: Markov decision process (MDP)

An MDP is a tuple (S, A, P, R, γ). Our running example:

		In the gridworld
States	S	12 cells; G and X are terminal
Actions	A	up, down, left, right
Transitions	$P(s' \mid s, a)$	move one cell (into a wall: stay)
Rewards	$R(s, a)$	$+1$ at G, -1 at X, else -0.02
Discount	γ	0.95



- **Markov property**: the next state depends only on the current state and action, not the full history:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t)$$

A special case: multi-armed bandits

- K arms (actions), $A = \{1, \dots, K\}$; arm k gives a reward drawn from an **unknown** distribution with mean μ_k
- Each round t: pick an arm $a_t \in A$, observe *only* the reward for that arm – **bandit feedback**
- One state: actions do not cause state transitions
- **Contextual bandit**: first observe a context x_t (e.g. user features); the best arm depends on x_t, but actions still do not change future contexts



Each slot machine ("one-armed bandit") is an arm. Photo: Yamaguchi-sensei, Wikimedia Commons, CC BY-SA 3.0.

Regret: the cost of not knowing μ_k in advance

$$\mathrm{Reg}(T) = T \cdot \mu^* - \mathbb{E}\Big[\sum_{t=1}^{T} r_t\Big], \qquad \mu^* = \max_k \mu_k$$

Exploration–Exploitation Dilemma

Exploit

keep choosing the **best-looking** option so far

vs.

Explore

try **other** options to reduce uncertainty about them

Example: ε-greedy

- With probability $1 - \varepsilon$: **exploit** – pick the best-looking option.
- With probability ε: **explore** – pick an option at random.

In RL the dilemma is harder: a reward may be *many steps* away, so the agent must explore whole trajectories, not single actions.

Sutton & Barto (2018), Ch. 2.

Policies and returns

- **Policy** $\pi(a \mid s)$: how the agent acts.
- **Return**: discounted sum of future rewards

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- **Worked example**: a 3-step episode in our gridworld, rewards $-0.02, -0.02, +1$:

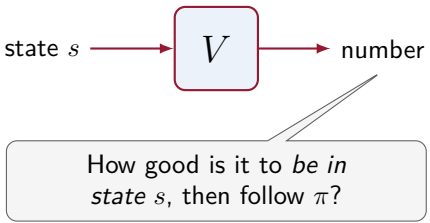
$$G_0 = -0.02 + 0.95(-0.02) + 0.95^2(1) = 0.8635$$

- **Objective**: find π maximising $J(\pi) = \mathbb{E}_\pi[G_0]$.

Sutton & Barto (2018), §3.3.

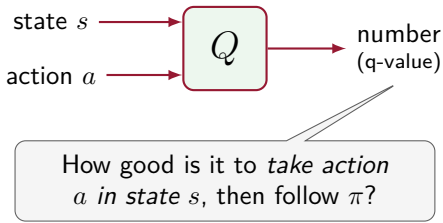
Value functions

State value function $V^\pi(s)$



$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

State-action value function $Q^\pi(s, a)$



$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

If we knew the optimal $Q^*(s, a) = \max_\pi Q^\pi(s, a)$, acting optimally is a lookup:
$\pi^*(s) = \arg\max_a Q^*(s, a)$.

Sutton & Barto (2018), §3.5–3.6.

The Bellman equation: the one idea to remember

The return decomposes recursively, $G_t = r_{t+1} + \gamma G_{t+1}$, so values do too:

Bellman equations

$$Q^\pi(s,a) = \mathbb{E}_\pi\big[\, r_{t+1} + \gamma Q^\pi(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a \,\big] \qquad \text{(expectation)}$$

$$Q^*(s,a) = \mathbb{E}\big[\, r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \,\big] \qquad \text{(optimality)}$$

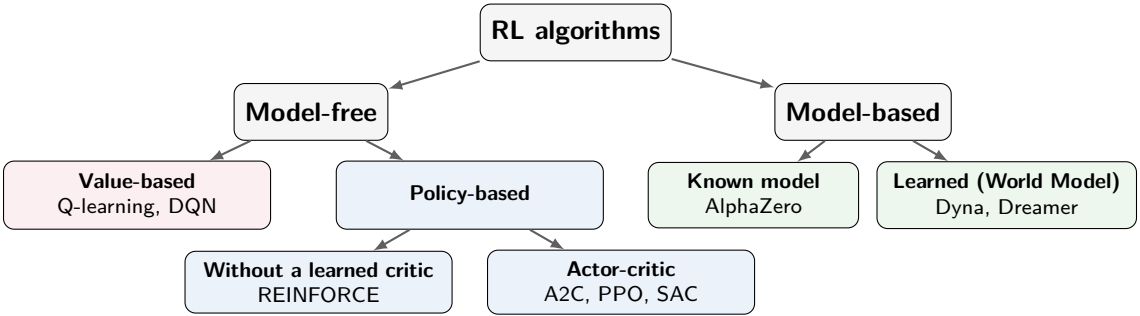
- "Value now = reward now + discounted value next."
- The only difference: the next action follows π (*average*) vs. the best next action (*max*).

Sutton & Barto (2018), §3.5–3.6.

Roadmap

1. The RL problem: MDPs, values, Bellman
2. **Three families of algorithms: value, policy, model**
3. RL meets large language models
4. RL in practice and open questions

The landscape of RL algorithms



Cross-cutting distinctions you will meet in papers

- **On-policy** (learn from the current policy's data) vs. **off-policy** (learn from data generated by another policy).
- **Online** (interact while learning) vs. **offline** (learn from a fixed logged dataset only).

Value-based: Q-learning

Turn the Bellman optimality equation into a sampled update:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \overbrace{\big(\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{TD target}} - Q(s,a)\big)}^{\text{TD error}}$$

- **TD (temporal-difference) learning**: update after *every step*, towards a target built from our own current estimate (*bootstrapping*) – no need to wait for the episode to end.
- **Off-policy**: learns Q^* whatever policy collected the data; explore with e.g. ε-greedy.

Watkins & Dayan (1992); Sutton & Barto (2018), §6.5.

From Q-learning to DQN

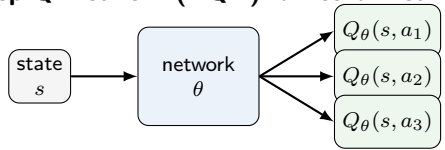
Q-learning: a table

	a_1	a_2	a_3
s_1	0.2	0.8	0.1
s_2	0.5	0.3	0.9
$\vdots$		$\vdots$	

one entry per (s, a) – impossible for huge state spaces

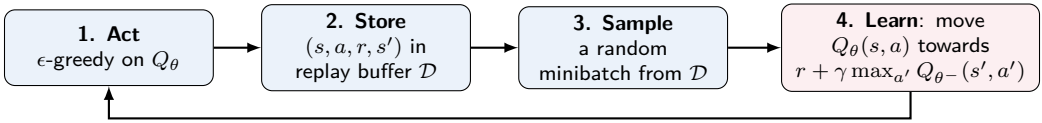
replace by

Deep Q-Network (DQN): a neural network



similar states share what is learned

Training loop



Mnih et al. (2015), *Nature*; Sutton & Barto (2018), §6.5.

Policy-based: policy gradients

Parameterise the policy directly, $\pi_\theta(a \mid s)$, and do gradient *ascent* on $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$.

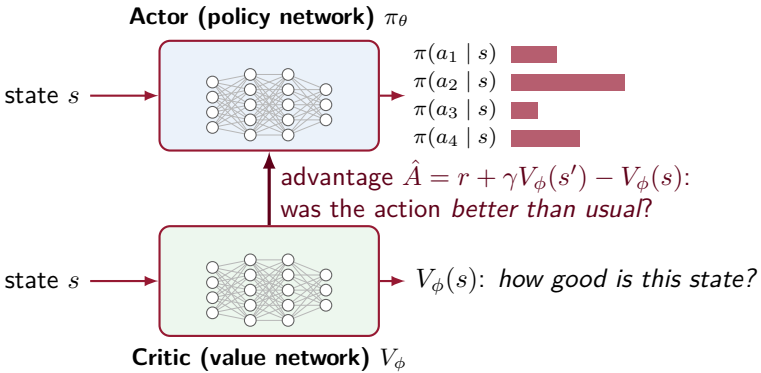
Policy gradient theorem (REINFORCE)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\Big]$$

- Intuition: make actions that led to *high return* more likely. In practice, subtract a baseline: "better than expected?"
- Compare supervised learning: maximise $\log p_\theta(y \mid x)$. Policy gradient = maximum likelihood on your *own* samples, **weighted by how well they did**.
- Handles continuous and stochastic actions; but on-policy and high-variance.

Williams (1992); Sutton et al. (2000); Sutton & Barto (2018), §13.2–13.4.

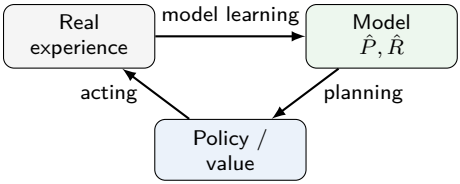
Actor-critic and PPO: today's workhorse



- **PPO** (Proximal Policy Optimisation): *limits how far the policy moves* per update – robust default; used for RLHF (InstructGPT/ChatGPT).

Sutton & Barto (2018), §13.5; Schulman et al. (2017).

Model-based RL: learn a world model, then plan



What does the model learn?

The two unknown parts of the MDP:

- $\hat{P}(s' \mid s, a)$: *where do I end up if I take a in s?*
- $\hat{R}(s, a)$: *what reward do I get?*

Together: a learned **simulator** of the environment.

How do we learn it? Supervised learning on observed transitions (s, a, r, s'):

- **Tabular**: count and average, $\hat{P}(s' \mid s, a) = \frac{N(s,a,s')}{N(s,a)}$; $\hat{R}(s, a) =$ mean observed reward.
- **Neural network**: predict s' and r from (s, a), minimising the prediction error.
- **Latent world model**: for pixels, predict the next compact latent state z and the reward, not the next image.

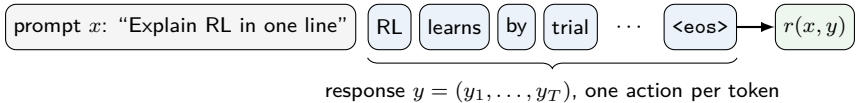
Examples: **Dyna** (tabular model); **Dreamer** (latent world model, learns "in imagination").

Sutton & Barto (2018), §8.1–8.2; Hafner et al. (2025).

Roadmap

1. The RL problem: MDPs, values, Bellman
2. Three families of algorithms: value, policy, model
3. **RL meets large language models**
4. RL in practice and open questions

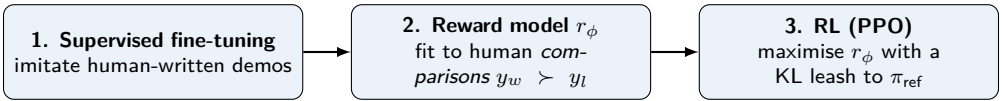
Text generation is an MDP



- **State** $s_t = (x, y_{<t})$; **action** $a_t = y_t$, the next token ($|A| \approx 10^5$); **policy** $=$ the LLM $\pi_\theta(y_t \mid x, y_{<t})$.
- **Transitions** are deterministic and known: once the policy *chooses* token y_t, the next state is simply $(x, y_{\leq t})$ – the randomness is in the policy's choice, not in the environment.
- **Reward** usually arrives only at the end: a human preference, a learned reward model, or a checker (unit tests, maths answer).

RL from human feedback (RLHF)

In RLHF, the **reward** $r(x, y)$ is a learned model r_ϕ of human preferences.



- The reward model learns to score answers humans prefer higher.
- RL objective: high reward, but stay close to the original model:

$$\max_\theta \; \mathbb{E}_{x,\; y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x,y)\big] - \beta\, D_{\mathrm{KL}}\big(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big)$$

- Why the KL term? r_ϕ is only a *proxy*: push too hard and the policy exploits its errors (**reward over-optimisation**).

Christiano et al. (2017); Ouyang et al. (2022); Gao et al. (2023).

Beyond RLHF: RL with verifiable rewards (RLVR)

- **Reward from a checker**, not a learned model: correct maths answer, passing unit tests.
- Harder to game than RLHF's learned reward model r_ϕ, which is only a proxy for human preferences.
- **GRPO** (Group Relative Policy Optimisation): sample several answers per prompt; score each against the *group average* – no critic needed.
- Long chain-of-thought "reasoning" behaviour emerged from this kind of training (DeepSeek-R1).

Lambert et al. (2024); Shao et al. (2024); DeepSeek-AI (2025).

Roadmap

1. The RL problem: MDPs, values, Bellman
2. Three families of algorithms: value, policy, model
3. RL meets large language models
4. **RL in practice and open questions**

RL in practice: the toolkit

The environment loop (Gymnasium)

```
import gymnasium as gym
env = gym.make("CartPole-v1")
obs, info = env.reset(seed=0)
for t in range(10_000):
    a = agent.act(obs)                    # your policy
    obs2, r, term, trunc, info = env.step(a)
    agent.update(obs, a, r, obs2, term)
    obs = obs2
    if term or trunc:
        obs, info = env.reset()
```

Or a tested implementation in two lines

```
from stable_baselines3 import PPO
PPO("MlpPolicy", "CartPole-v1").learn(100_000)
```

Useful libraries

- **Gymnasium** / **PettingZoo**: single- / multi-agent environments
- **Stable-Baselines3**: reliable reference algorithms
- **CleanRL**: single-file implementations, easy to read and modify
- **TRL** (Hugging Face): PPO, DPO, GRPO for LLMs
- JAX stacks (PureJaxRL, Brax): environments run on the GPU, orders of magnitude faster

Which kind of RL fits *your* problem?

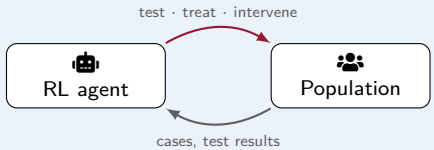
Do your actions change future states?	**No** → (contextual) bandits: the simplest RL, with theory
Do you have demonstrations of good behaviour?	**Yes** → imitate first, then fine-tune with RL (as for LLMs)
Is real interaction cheap (e.g. a simulator)?	**No** → offline RL from logged data, or model-based RL
Is the reward easy to write down and check?	**No** → learn the reward from preferences or demos (RLHF, IRL)

Open research questions

- **Sample efficiency**: agents still need far more experience than people. Can world models and better exploration close the gap?
- **Offline RL**: learning good policies from logged data alone, without new interaction – and knowing when to trust them.
- **Reward design & alignment**: where do rewards come from? Preferences, verifiers and proxies that cannot be gamed.
- **Generalisation & adaptation**: policies that transfer across tasks and keep adapting when the world changes.

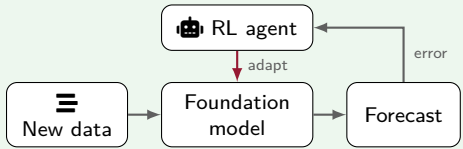
Open PhD positions

RL for infectious disease surveillance and control



Where and whom to test, how to control outbreaks, and **personalised interventions** for each individual.

Test-time adaptation of time-series foundation models with RL



Using RL to adapt a pretrained time-series foundation model to **new data at test time**.

Interested? Come and talk to me after the seminar!

Summary

1. The problem
Learn to act from *evaluative, delayed* feedback. MDPs formalise it; the *Bellman equation* drives almost every algorithm.

2. The algorithms
Value-based (Q-learning, DQN), *policy-based* (policy gradients), *model-based* (world models).

3. RL meets large language models
Fine-tuning LLMs with RLHF and with verifiable rewards.

4. In practice & open questions
Pick the RL setting that fits your data and reward. Open: sample efficiency, offline RL, reward design, generalisation.

Reading and resources

Core reading

- Sutton & Barto, *Reinforcement Learning: An Introduction* (2nd ed., 2018), Ch. 1, 3, 6 and 13. Free online.

Going further

- OpenAI, *Spinning Up in Deep RL* – practical introduction with code. spinningup.openai.com
- K. Murphy, *Reinforcement Learning: A Comprehensive Overview* (arXiv, 2024) – up-to-date survey.
- N. Lambert, *Reinforcement Learning from Human Feedback* (online book). rlhfbook.com
- Albrecht, Christianos & Schäfer, *Multi-Agent Reinforcement Learning* (MIT Press, 2024). marl-book.com
- Lattimore & Szepesvári, *Bandit Algorithms* (2020) – exploration with theory.

References

Christiano et al. (2017). Deep RL from human preferences. *NeurIPS*.

DeepSeek-AI (2025). DeepSeek-R1: incentivizing reasoning capability in LLMs via RL. *arXiv:2501.12948*.

Degrave et al. (2022). Magnetic control of tokamak plasmas through deep RL. *Nature*.

Gao et al. (2023). Scaling laws for reward model overoptimization. *ICML*.

Hafner et al. (2025). Mastering diverse control tasks through world models. *Nature*.

Lambert et al. (2024). Tülu 3: pushing frontiers in open LM post-training. *arXiv:2411.15124*.

Mnih et al. (2015). Human-level control through deep RL. *Nature*.

Ouyang et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS*.

Schrittwieser et al. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*.

Schulman et al. (2017). Proximal policy optimization algorithms. *arXiv:1707.06347*.

Shao et al. (2024). DeepSeekMath. *arXiv:2402.03300*.

Silver et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*.

Silver et al. (2018). A general RL algorithm that masters chess, shogi, and Go through self-play. *Science*.

Sutton & Barto (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Sutton et al. (2000). Policy gradient methods for RL with function approximation. *NeurIPS*.

Watkins & Dayan (1992). Q-learning. *Machine Learning*.

Williams (1992). Simple statistical gradient-following algorithms. *Machine Learning*.

Thank you – questions?

Mengyan Zhang mengyan.zhang@bristol.ac.uk